

Handler:

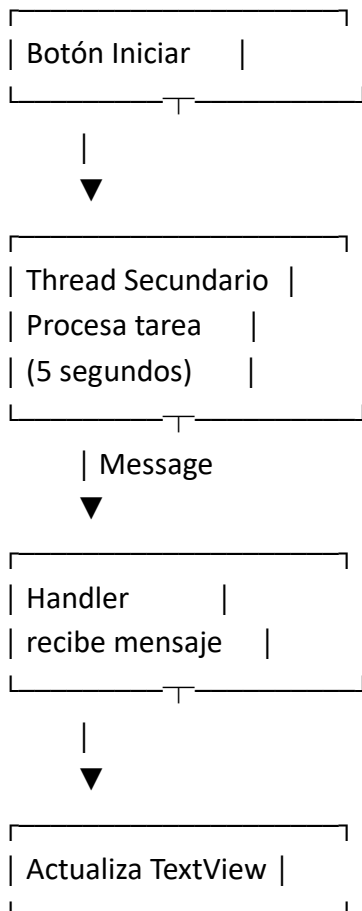
Handler con mensajes (Message). Handler con objetos Runnable

Objetivo

- *Qué es el hilo principal (UI Thread).*
- *Por qué no debemos actualizar la interfaz desde otros hilos.*
- *Cómo funciona un Handler.*
- *Cómo enviar información mediante objetos Message.*
- *Cómo programar tareas mediante objetos Runnable.*
- *Cómo observar el comportamiento mediante logs.*

Ejercicio 1: Handler usando Message

¿Qué vamos a simular? Un trabajador (Thread secundario) realiza una tarea de 5 segundos. Cuando termina, envía un mensaje al Handler del hilo principal. El Handler recibe el mensaje y actualiza la interfaz. **Esquema visual**



Layout XML

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <Button
        android:id="@+id/btnMensaje"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ejecutar Message"/>

    <Button
        android:id="@+id/btnRunnable"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ejecutar Runnable"
        android:layout_marginTop="20dp"/>

    <Button
        android:id="@+id/btnLog"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Generar Log"
        android:layout_marginTop="20dp"/>

    <TextView
        android:id="@+id/txtResultado"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="22sp"
        android:layout_marginTop="30dp"
        android:text="Esperando..."/>

</LinearLayout>
```

Código Kotlin

MainActivity.kt

```
package com.example.handlerdemo

import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.os.Message
import android.util.Log
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    private lateinit var txtResultado: TextView

    companion object {
        const val TAG = "MI_HANDLER"
        const val CODIGO_EXITO = 1
    }

    // Handler asociado al hilo principal
    private val miHandler = object : Handler(Looper.getMainLooper()) {

        override fun handleMessage(msg: Message) {

            when(msg.what){

                CODIGO_EXITO -> {

                    val texto = msg.obj as String

                    txtResultado.text = texto

                    Log.d(TAG, "Mensaje recibido: $texto")
                }
            }
        }
    }
}
```

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  
    txtResultado = findViewById(R.id.txtResultado)  
  
    val btnMensaje = findViewById<Button>(R.id.btnMensaje)  
    val btnRunnable = findViewById<Button>(R.id.btnRunnable)  
    val btnLog = findViewById<Button>(R.id.btnLog)
```

```
// EJEMPLO 1: MESSAGE
```

```
btnMensaje.setOnClickListener {  
  
    txtResultado.text = "Procesando..."  
  
    Thread {  
  
        Log.d(TAG, "Thread iniciado")  
  
        Thread.sleep(5000)  
  
        val mensaje = Message()  
  
        mensaje.what = CODIGO_EXITO  
        mensaje.obj = "Tarea finalizada con Message"  
  
        miHandler.sendMessage(mensaje)  
  
    }.start()  
}
```

```
// EJEMPLO 2: RUNNABLE
```

```
btnRunnable.setOnClickListener {  
  
    txtResultado.text = "Esperando Runnable..."  
  
    miHandler.postDelayed({
```

```

        txtResultado.text =
            "Runnable ejecutado después de 3 segundos"

        Log.d(TAG, "Runnable ejecutado")

    },3000)
}

// BOTON DE LOGEO

btnLog.setOnClickListener {

    Log.d(TAG,"Log DEBUG")
    Log.i(TAG,"Log INFO")
    Log.w(TAG,"Log WARNING")
    Log.e(TAG,"Log ERROR")

    txtResultado.text = "Revisar Logcat"
}
}
}

```

Explicación

Parte 1: El hilo principal

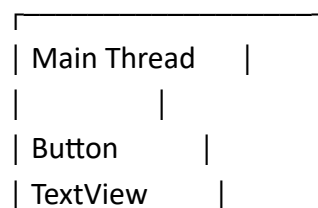
Cuando Android inicia una aplicación crea automáticamente:

Main Thread (UI Thread)

Este hilo es responsable de:

Dibujar botones
Dibujar TextView
Gestionar clics
Gestionar eventos táctiles

Visualmente:



```
| RecyclerView |  
└──────────┘
```

Problema

Si hacemos una tarea pesada:

```
Thread.sleep(5000)
```

directamente en el Main Thread:

```
btn.setOnClickListener {
```

```
    Thread.sleep(5000)
```

```
}
```

la aplicación se congela.

Visualmente:

Main Thread

```
|  
├── Dibujar UI  
├── Eventos  
└── Esperar 5 segundos
```

Todo queda bloqueado.

Solución: Thread secundario

Creamos un hilo aparte.

```
Thread {
```

```
}.start()
```

Visualmente:

Main Thread

```
|  
└── UI
```

Worker Thread

```
|  
└── Trabajo pesado
```

¿Por qué aparece Handler?

Porque Android NO permite que un Thread secundario modifique la interfaz.

Esto es incorrecto:

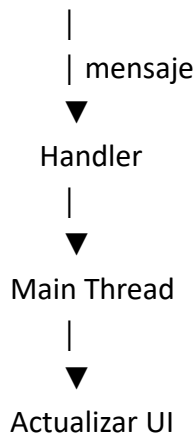
```
Thread {  
  
    txtResultado.text = "Hola"  
  
}
```

Produce errores.

Handler como mensajero

El Handler actúa como un intermediario.

Thread Secundario



Analizando el Message

Creamos:

```
val mensaje = Message()
```

El objeto Message es un sobre que transporta información.

Campo what

```
mensaje.what = CODIGO_EXITO
```

Sirve como identificador.

Ejemplo:

1 = Éxito

2 = Error

3 = Advertencia

Campo obj

`mensaje.obj = "Proceso terminado"`

Transporta datos.

Visualmente:

Message

|

|— what = 1

|— obj = "Proceso terminado"

Recepción del mensaje

`override fun handleMessage(msg: Message)`

Es el buzón del Handler.

Message llega

|



`handleMessage()`

|



Procesar información

¿Qué hace `sendMessage`?

`miHandler.sendMessage(mensaje)`

Envía el mensaje al Handler.

Visualmente:

Worker Thread

|

| `sendMessage()`



Handler

|



Main Thread

Ejemplo Runnable

Ahora no enviamos datos. Solo queremos ejecutar algo después de un tiempo.

`miHandler.postDelayed(...)`

¿Qué es Runnable?

Es un bloque de código ejecutable.

```
Runnable {
```

```
}
```

o en Kotlin:

```
{
```

```
}
```

Flujo visual

Handler

|

|— Esperar 3 segundos

|



Runnable

|



Actualizar TextView

post()

Ejecuta inmediatamente.

```
handler.post {
```

```
}
```

Visualmente:

Handler

|



Runnable

postDelayed()

Ejecuta más tarde.

```
handler.postDelayed({
```

```
},3000)
```

Visualmente:

0 seg

|



Espera

|



3 seg

|



Runnable

Comparación Message vs Runnable

Característica	Message	Runnable
Transporta datos	Sí	No
Tiene identificador	Sí (what)	No
Comunicación entre hilos	Excelente	Limitada
Tareas temporizadas	Posible	Muy cómoda
Fácil para principiantes	Media	Alta

Explicación del botón de logeo

Log.d(TAG,"Log DEBUG")

Muestra mensajes de depuración.

Tipos de logs

Log.d()

Debug

Log.i()

Información

Log.w()

Advertencia

Log.e()

Error

Visualmente en Logcat:

D/MI_HANDLER: Log DEBUG

I/MI_HANDLER: Log INFO

W/MI_HANDLER: Log WARNING

E/MI_HANDLER: Log ERROR

¿Cuándo usar Message y cuándo Runnable?

Message

Cuando un hilo necesita enviar información a otro hilo.

Ejemplo:

Descargar archivo

→ enviar resultado

→ actualizar interfaz

Runnable

Cuando solo queremos ejecutar una tarea.

Ejemplo:

Mostrar anuncio

después de 5 segundos

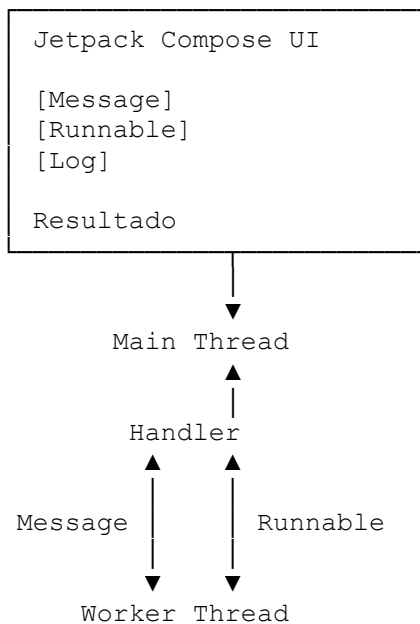
Conclusión: Handler es un mecanismo de comunicación entre hilos en Android.

Thread -----> Handler -----> UI
Puede trabajar de dos formas:
1. Message
Envía datos.
Ideal para comunicación entre hilos.
2. Runnable
Ejecuta código.
Ideal para tareas programadas o retardadas.

Objetivo de la práctica

- Uso de Handler con Message.
- Uso de Handler con Runnable.
- Uso de un hilo secundario (Thread).
- Actualización de estados Compose.
- Visualización de logs en Logcat.

Arquitectura visual del ejercicio



Interfaz esperada

EJEMPLO HANDLER

```
[ Ejecutar Message ]
```

```
[ Ejecutar Runnable ]
```

[Generar Logs]

Resultado:
Esperando...

MainActivity.kt

```
package com.example.handlercompose

import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.os.Message
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.Text
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {

    companion object {
        const val TAG = "HANDLER_COMPOSE"
        const val CODIGO_EXITO = 1
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            PantallaHandler()
        }
    }

    @Composable
    fun PantallaHandler() {

        var resultado by remember {
            mutableStateOf("Esperando...")
        }

        // Handler asociado al hilo principal
        val handler = remember {

            object : Handler(Looper.getMainLooper()) {

                override fun handleMessage(msg: Message) {

                    when (msg.what) {

                        CODIGO_EXITO -> {

                            resultado = msg.obj as String

                            Log.d(
                                TAG,
                                "Mensaje recibido: ${msg.obj}"
                            )
                        }
                    }
                }
            }
        }
    }
}
```

```

        )
    }
}

}

Scaffold {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(it)
            .padding(20.dp),

        horizontalAlignment =
            Alignment.CenterHorizontally,

        verticalArrangement =
            Arrangement.spacedBy(16.dp)
    ) {

        Text(
            text = "EJEMPLO HANDLER",
            style = MaterialTheme.typography.headlineMedium
        )

        //-----
        // BOTÓN MESSAGE
        //-----

        Button(

            onClick = {

                resultado = "Procesando..."

                Thread {

                    Log.d(
                        TAG,
                        "Thread iniciado"
                    )

                    Thread.sleep(5000)

                    val mensaje = Message()

                    mensaje.what = CODIGO_EXITO

                    mensaje.obj =
                        "Proceso terminado usando Message"

                    handler.sendMessage(mensaje)

                }.start()
            }

        ) {
            Text("Ejecutar Message")
        }
    }
}

```

```

//-----
// BOTÓN RUNNABLE
//-----

Button(

    onClick = {

        resultado =
            "Esperando Runnable..."

        handler.postDelayed({

            resultado =
                "Runnable ejecutado tras 3 segundos"

            Log.d(
                TAG,
                "Runnable ejecutado"
            )

        }, 3000)

    }

) {
    Text("Ejecutar Runnable")
}

//-----
// BOTÓN LOGS
//-----

Button(

    onClick = {

        Log.d(TAG, "DEBUG")
        Log.i(TAG, "INFO")
        Log.w(TAG, "WARNING")
        Log.e(TAG, "ERROR")

        resultado =
            "Revisa Logcat"

    }

) {
    Text("Generar Logs")
}

//-----
// RESULTADO
//-----

Card(
    modifier = Modifier.fillMaxWidth()
) {

    Column(
        modifier = Modifier.padding(20.dp)
    ) {

```


2. ¿Por qué seguimos necesitando Handler?

Aunque Compose sea moderno, Android sigue teniendo:

Main Thread

para:

- Dibujar la interfaz
- Gestionar eventos
- Gestionar recomposición

3. El Thread secundario

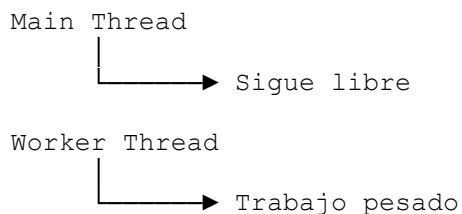
Cuando pulsamos:

Ejecutar Message

se crea:

```
Thread {  
}.start()
```

Visualmente:



4. Simulación de trabajo

```
Thread.sleep(5000)
```

Representa:

Descarga
Consulta BD
Llamada API
Procesamiento

Uso de Message

Crear el mensaje

```
val mensaje = Message()
```

Piensa en un sobre.

```
Message
```

Campo what

```
mensaje.what = CODIGO_EXITO
```

Identifica el tipo.

```
1 = éxito  
2 = error  
3 = aviso
```

Campo obj

```
mensaje.obj = "Proceso terminado"
```

Transporta información.

```
Message  
├──  
└── what = 1  
    obj = "Proceso terminado"
```

Enviar mensaje

```
handler.sendMessage(mensaje)
```

Visualmente:

```
Worker Thread  
  │  
  ▼  
Message  
  │  
  ▼  
Handler  
  │  
  ▼  
Main Thread
```

Recepción

```
override fun handleMessage(msg: Message)
```

Es el buzón.

Handler
↓
handleMessage()

Cambio de estado Compose

```
resultado = msg.obj as String
```

Al cambiar el estado:

Estado cambia
↓
Compose recompone
↓
Pantalla actualizada

Uso de Runnable

Ahora no enviamos datos.

Solo ejecutamos una acción.

```
handler.postDelayed({  
}, 3000)
```

Flujo

Handler
↓
Espera 3 segundos
↓
Runnable
↓
Actualiza estado

Runnable

Es un bloque ejecutable.

```
{  
    resultado = "Hola"  
}
```

Visualmente:

```
Runnable
├── instrucción 1
├── instrucción 2
└── instrucción 3
```

Diferencia visual

Message

```
Worker Thread
  ↓
Message
  ↓
Handler
```

Transporta información.

Runnable

```
Handler
  ↓
Runnable
```

Transporta comportamiento.

Botón de logs

```
Log.d(TAG, "DEBUG")
```

Permite enseñar Logcat.

Salida esperada

```
D/HANDLER_COMPOSE: DEBUG
I/HANDLER_COMPOSE: INFO
W/HANDLER_COMPOSE: WARNING
E/HANDLER_COMPOSE: ERROR
```

Comparación

Aspecto	Message	Runnable
Envía datos	Sí	No
Tiene identificador	Sí (what)	No
Comunicación entre hilos	Excelente	Limitada
Retrasos temporales	Posible	Muy sencillo
Caso típico	Resultado de una tarea	Ejecutar una acción futura

Concepto clave

Compose NO elimina los hilos.
Compose NO elimina Handler.
Compose sigue usando Main Thread.

La diferencia es que ahora:

XML

↓

Modificar Views

Compose

↓

Modificar Estados

↓

Recomposición automática

Handler + Thread + Message + Runnable + Estado Compose + Recomposition,
--